Université de Montpellier — Faculté des Sciences
Master Informatique

UE HAI716 — Projet de semestre

Le robot de réconfort

Un agent à perception locale dans un environnement inconnu

Rapport du projet 1

Auteurs

Skander SAAFI
Abdellah DORBANE

Résumé. Ce rapport présente le robot de réconfort que nous avons développé en Python. Le robot découvre les murs de l'appartement en se déplaçant, trouve l'émotion d'un message grâce à un dictionnaire, cherche un objet dans une armoire, puis l'apporte au résident. Sur les cinq scénarios fournis, les 29 demandes sont livrées.

Année universitaire 2026–2027

1 Introduction

Une résidence est équipée d'un robot de service. Chaque résident peut lui envoyer un court message. Le robot doit alors trouver l'émotion du message grâce à un dictionnaire, chercher dans une armoire l'objet de réconfort qui correspond, puis l'apporter au résident. La difficulté principale est que le robot ne connaît pas le plan de l'appartement : à chaque tour, il ne voit que les quatre cases autour de lui. Il ne sait pas non plus ce que contient un casier tant qu'il ne l'a pas regardé. Nous avons écrit ce robot en Python 3.12.

Ce rapport présente l'organisation du code (section 2), la façon dont le robot change de chemin (section 3), la façon dont il choisit le casier (section 4) et un cas où notre robot échoue (section 5).

2 Architecture de l'agent

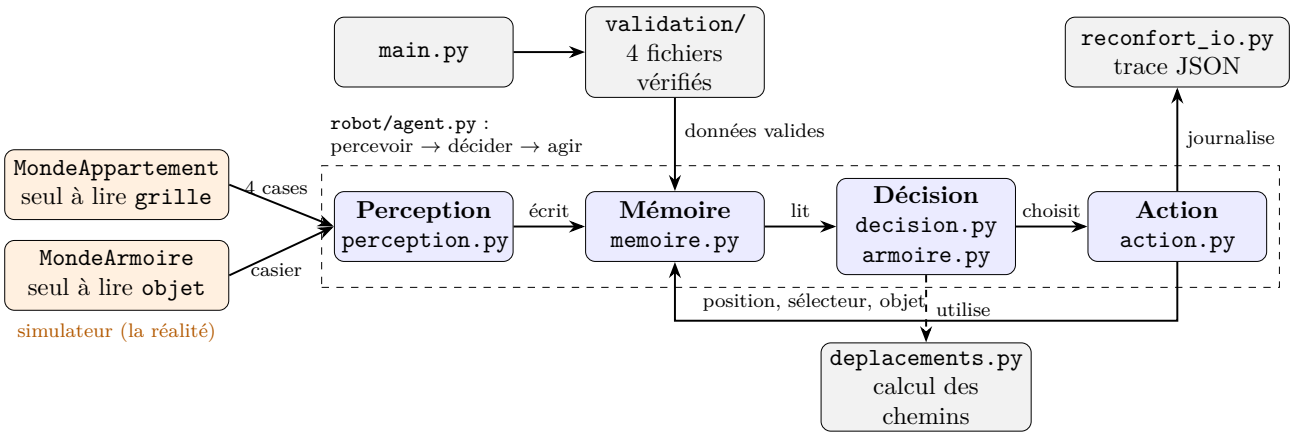


Figure 1 – Architecture du robot. En bleu, les quatre parties de l'agent ; en orange, le simulateur, seul à connaître la réalité.

La figure 1 montre comment le code est organisé. À chaque tour, le robot fait toujours la même chose : il regarde autour de lui, il met à jour ce qu'il sait, il décide, puis il agit. Nous avons séparé ces quatre étapes selon une règle simple : **chaque partie n'a accès qu'à ce dont elle a besoin**.

— La **perception** est la seule partie qui communique avec le monde. Elle donne au robot les quatre cases autour de lui, et le contenu du casier quand il est devant l'armoire. Seule exception : pendant la fouille, `armoire.py` regarde lui-même le casier devant le sélecteur. Le plan complet et le contenu de l'armoire ne sont connus que du simulateur (`monde/`).

— La **mémoire** garde ce que le robot a appris : sa carte (avec `?` pour les cases inconnues), sa position, l'objet qu'il porte et les casiers déjà vus. Elle est gardée d'une demande à l'autre.

— La **décision** choisit la prochaine action en lisant seulement la mémoire, ce qui la rend facile à tester.

— L'**action** exécute ce qui a été décidé et l'écrit dans la trace.

3 Stratégie de replanification

Au départ, le robot ne connaît pas les murs. Il fait donc une supposition simple : **toute case qu'il n'a pas encore vue est libre** [1]. Ensuite, il ne garde pas un plan fixe : **il recalcule son chemin à chaque pas**. À chaque tour :

1. il regarde les quatre cases autour de lui et les note dans sa carte ;
2. il calcule le plus court chemin vers sa cible, avec ce qu'il sait (parcours en largeur) ;
3. il fait seulement le premier pas de ce chemin, puis recommence.

Quand il découvre un mur sur sa route, il n'a rien de spécial à faire : au tour suivant, le nouveau calcul tient compte de ce mur et trouve un autre chemin. Si aucun chemin n'est trouvé, même en supposant libres toutes les cases inconnues, alors il n'en existe pas dans la réalité : la cible est inaccessible. Le robot le vérifie avant de prendre un objet, pour ne pas prendre un objet qu'il ne pourra pas donner.

Coût. Le calcul reste instantané sur ces cartes. Le vrai coût, ce sont les détours vers des passages bouchés : entre 8 % et 30 % de pas en plus que si le robot connaissait les murs. Nous avons gardé cette méthode car elle est simple et toujours juste.

4 Choix du casier dans l'armoire

Notre robot applique la règle de repli de l'énoncé (section 6.2), qui s'appuie sur la roue de Plutchik [2]. Une fois l'émotion et l'intensité connues :

1. il garde les 15 casiers dont l'émotion est à une distance de 2 ou moins sur la roue ;
2. il les range dans l'ordre imposé : distance sur la roue, puis écart d'intensité, puis intensité la plus faible, puis sens horaire ;
3. il déplace le sélecteur casier par casier et prend l'objet du **premier casier garni** de cette liste.

Le robot ne connaît le contenu d'un casier qu'en passant devant, et il retient les casiers déjà vus d'une demande à l'autre. S'il croise un casier garni moins bien placé dans la liste, il ne le prend pas. Si les 15 casiers sont vides, la demande échoue avec le motif « armoire vide ». Par exemple, pour `surprise/forte` (scénario 3), la colonne surprise est vide : le robot regarde d'abord les trois casiers surprise, puis prend le téléphone (`tristesse/forte`), et la trace indique le repli `voisine_1`.

Piste d'amélioration. Lors de nos tests, nous avons remarqué qu'une émotion proche sur la roue ne donne pas toujours l'objet le plus adapté : pour une personne triste, le désodorisant (dégoût) passe avant la veilleuse (peur). Une piste serait d'indiquer dans le fichier d'armoire, pour chaque émotion, la liste des émotions de remplacement acceptables. Il faudrait alors adapter `armoire.json`, sa validation et la fonction `casiers_candidats`.

5 Un échec de notre robot

Ce qui se passe. Nous avons dessiné une petite carte où le résident R2 est enfermé entre quatre murs. De loin, le robot ne peut pas le voir. Quand il arrive à l'armoire, il pense donc pouvoir rejoindre R2 et il prend la couverture. Sur le chemin, il découvre les murs et comprend que R2 est inaccessible : jusque-là, c'est le bon comportement. Le problème vient ensuite. Le robot ne peut pas reposer la couverture, car aucune action ne le permet. À la demande suivante, R1 est joyeux : le robot ne peut pas prendre un deuxième objet, et la demande échoue avec le motif « armoire vide », alors que le bon casier est plein.

Pourquoi. Le robot ne connaît que les murs qu'il a déjà vus. Quand il est à l'armoire, il n'a pas encore vu ceux qui entourent R2 : sur sa carte, ces cases sont encore inconnues, donc supposées libres, et un chemin vers R2 semble exister. Le robot ne peut savoir que R2 est enfermé qu'en s'approchant de lui, donc après avoir pris l'objet. Comme il ne porte qu'un objet à la fois, il ne peut plus en prendre un autre ensuite.

Ce qu'on pourrait faire. Vérifier le chemin jusqu'au résident avant de prendre l'objet : cela coûte des pas, mais évite de rester bloqué.

6 Conclusion

Notre robot respecte la perception locale, son code est séparé en quatre parties claires, et il livre toutes les demandes des scénarios fournis. L'échec étudié montre qu'un robot qui ne voit pas tout peut se tromper au mauvais moment. Pour le projet 2, nous réutiliserons cette organisation avec plusieurs robots.

Références

[1] Projet — Le robot de réconfort, énoncé du projet de semestre, Master Informatique, UE HAI716.
[2] R. Plutchik, *Emotion : A Psychoevolutionary Synthesis*, Harper & Row, 1980.